AIOPS-99: Series Summary

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 8 of 8 | **Created:** May 2026 | **Last Updated:** 10/23/2026

Overview

Closing notebook for the AIOPS series. Recaps the seven chapters, indexes the canonical DQL queries, lists the cross-series pointers, and surfaces the next steps for an AIOps initiative.

Audience: Anyone returning to the series after first read; anyone planning a follow-on initiative.

Table of Contents

1. What This Series Covered
2. Canonical DQL Queries
3. Cross-Series Pointers
4. Where to Go Next
5. Source Currency

Prerequisites

Requirement	Details
Reading order	Best read after at least AIOPS-01; selectively after the chapters you've already used

Requirement	Details
No DQL execution required	This notebook is reference material; queries are reproduced as text

1. What This Series Covered

Chapter	Subject	One-line takeaway
AIOPS-01	Overview	Three AI categories — Causal, Predictive, Generative — over Smartscape and Grail
AIOPS-02	Anomaly Detection	Five mechanisms; pick by metric character, not by habit
AIOPS-03	Problems & RCA	`fetch dt.davis.problems`; topology completeness gates RCA quality
AIOPS-04	Davis CoPilot / Dynatrace Assist	Generative AI for DQL, summaries, explanations; not a replacement for tenant-specific knowledge
AIOPS-05	AI Models	Causal correlation, predictive baselines, generative LLM — three different model families
AIOPS-06	Integrations & Agentic	Workflow AI tasks, MCP server, AutomationEngine; agentic in suggest-mode first
AIOPS-07	Putting It Together	Detect → Investigate → Remediate; capacity, incident, regression scenarios

2. Canonical DQL Queries

Active problem feed (the most-used query in the series):

```
fetch dt.davis.problems, from:-2h
| filter event.status == "ACTIVE"
| sort event.start desc
| fields display_id, event.name, event.category, event.start,
         root_cause_entity_name, affected_entity_ids
| limit 50
```

Problem category rollup — 7 days:

```
fetch dt.davis.problems, from:-7d
| summarize problem_count = count(), by:{event.category}
| sort problem_count desc
```

Problem severity rollup — 7 days: a different axis from category, and the one AIOPS-03 § 5 treats as
distinct. Note `event.severity` is being **promoted to** `stable` in Semantic Dictionary 1.348 (released
08/25/2026), across Davis events, Davis event reports and Davis problems. It was `experimental`,
which is why earlier revisions of this summary said to use it for triage but not to hard-code an
integration against it — that caution is what the promotion lifts.

Confirm before you rely on it: the dictionary ships with the platform version, and a live tenant
checked on 09/09/2026 still returned `experimental`, so 1.348 had not reached it. Until it does,
the original caution stands.

SaaS 1.348 (rollout start 10/12/2026, staged tenant rollout) also changes the value itself: *"Davis events and problems no longer default event.severity to 3."* Once that reaches your tenant,
an unset severity is null — the query below tests for it first and labels it `not set`, rather than
letting it fall through to `5 - informational`.

> **Sources:** What's new in Dynatrace SaaS 1.348 (DT docs), Semantic Dictionary changelog 1.348 (DT docs), Standardized event severity (DT docs) — *"If an event source doesn't set severity, the field is empty."*

```
fetch dt.davis.problems, from:-7d
| fieldsAdd severity_label = if(isNull(event.severity), "not set",
                            else: if(event.severity == 1, "1 - critical",
                            else: if(event.severity == 2, "2 - major",
                            else: if(event.severity == 3, "3 - minor",
                            else: if(event.severity == 4, "4 - warning",
else: "5 - informational")))))
| summarize problem_count = count(), by:{event.severity, severity_label}
| sort event.severity asc
```

MTTR by category — 30 days:

```
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| fieldsAdd duration = event.end - event.start
| summarize {
    median_mttr = percentile(duration, 50),
    p95_mttr    = percentile(duration, 95),
    problem_count = count()
  },
  by:{event.category}
| sort problem_count desc
```

Raw signal volume by category:

```
fetch dt.davis.events, from:-1h
| summarize signal_count = count(), by:{event.category}
| sort signal_count desc
```

Signal-to-problem compression trend: count problem-eligible signals only — INFO / WARNING
events never open a problem, and PROBLEM_UPDATE records are lifecycle updates, not signals.

```
// Run both, compare:
fetch dt.davis.events, from:-7d
| filter in(event.category,
{"AVAILABILITY","ERROR","RESOURCE_CONTENTION","SLOWDOWN","CUSTOM_ALERT"})
| makeTimeseries signals = count(), interval:24h

fetch dt.davis.problems, from:-7d
| makeTimeseries problems = count(), interval:24h
```

Each query above is a text-only copy of one executed in AIOPS-01, AIOPS-03 or AIOPS-07.

3. Cross-Series Pointers

Where AIOPS hands off:

Series	Hand-off
WFLOW	Notification routing on active problems
AUTOM	Anomaly detection settings as code (Monaco / Terraform)
ADOPT	Observability maturity model (ADOPT-01); the AIOps stages are in AIOPS-07 §6
DASH	Problem-driven SLO and executive dashboards
K8S, DBMON, CLOUD	Domain-specific anomaly patterns layered on these mechanisms
OPLOGS, OPMIG	Pre-AI investment: clean log streams that AI can reason over

Where AIOPS is the canonical home:

- AI category model (Causal / Predictive / Generative)
- Davis problem and event data shape
- Davis analyzer family
- Dynatrace Assist surfaces and patterns
- MCP server tool catalog (problem and analyzer surface)

4. Where to Go Next

If you came here looking for a starting point, pick one based on where you are:

- **No detectors tuned yet** → AIOPS-02, then AUTOM-03 (Monaco) / AUTOM-04 (Terraform) to put settings in code
- **Problems flooding the on-call rotation** → AIOPS-03 to understand grouping; WFLOW for notification design; ADOPT-01 for maturity framing

- **Team not using Assist productively** → AIOPS-04; pair with a working session reviewing the prompt patterns
- **Considering BYO LLM or external AI in workflows** → AIOPS-05 (model boundaries) and AIOPS-06 (integration surfaces)
- **Building an end-to-end loop** → AIOPS-07 scenarios, then operationalize with the WFLOW patterns

Three high-value next initiatives most teams should consider:

1. **Audit topology completeness.** RCA quality is bounded by Smartscape coverage. Inventory untraced services and undeclared dependencies; treat each gap as a backlog item.
2. **Move detectors to config-as-code.** Shift the Anomaly Detection app from primary configuration tool to exploration tool. AUTOM-03 / AUTOM-04.
3. **Introduce one workflow with an AI task.** Start with the *Summarize Open Problems* pattern — high value, low risk, builds team familiarity with AI in workflows.

> **Custom alerts move to Settings (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and initiative 2 refers to that app.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

5. Source Currency

Last verified: **09/24/2026** (all DQL executed; citations checked).

Areas that are evolving fast in 2026 — re-check before relying on screenshots or specific UI flows:
- Dynatrace Assist surfaces (UI placement, conversation starters, action-suggesting features)
- Davis CoPilot agentic features (preview features change naming and scope frequently)

- Workflow AI task templates (new templates ship monthly)
- MCP server tool catalog (new tools added; existing tools sometimes renamed)

If a query stops returning expected results, validate against `mcp__dynatrace__execute-dql` first — the data model is more stable than the UI.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.